RAPPORT DE TRAVAUX PRATIQUES

TP 4 : Programmation Orientée Objet (POO) — Java

Encapsulation · Héritage · Abstraction · Polymorphisme

Module	Génie Logiciel
Encadrant	Pr. BE. ELBAGHAZAOUI
Réalisé par	Ikram El-MAZINI
Année universitaire	2025 / 2026

1. Introduction

Ce TP porte sur les concepts fondamentaux de la Programmation Orientée Objet (POO) en Java. À travers trois exercices progressifs, nous mettons en pratique les quatre piliers de la POO : l'encapsulation, l'héritage, l'abstraction et le polymorphisme. Les exercices couvrent la gestion d'un zoo, d'un système de transport et d'un système bancaire.

2. Exercice 1.1 — Gestion d'animaux dans un zoo

Ce premier exercice modélise un système de gestion d'animaux en utilisant une classe abstraite Animal et deux sous-classes Mammifere et Oiseau.

2.1 Partie 1 — Réponses théoriques

Q1. Encapsulation :

> L'encapsulation consiste à regrouper les données (attributs) et les méthodes qui les manipulent dans une même classe, en cachant les détails internes. Dans la classe Animal, l'attribut nom est déclaré private, ce qui interdit l'accès direct depuis l'extérieur. L'accès se fait uniquement via le getter getNom(), ce qui protège la donnée contre des modifications non contrôlées.

Q2. Héritage :

> L'héritage permet à une classe fille de réutiliser les attributs et méthodes d'une classe parente. Dans ce code, Mammifere extends Animal et Oiseau extends Animal illustrent ce principe. Les deux sous-classes héritent de l'attribut nom et du getter getNom(), et fournissent chacune leur propre implémentation de faireDuBruit().

Q3. Abstraction :

> L'abstraction consiste à définir une structure générale sans en préciser l'implémentation. La classe Animal est déclarée abstract, ce qui l'empêche d'être instanciée directement. La méthode faireDuBruit() est abstraite : elle impose un contrat à toutes les sous-classes qui doivent obligatoirement l'implémenter.

Q4. Polymorphisme :

> Le polymorphisme permet d'appeler la même méthode sur des objets de types différents, avec des comportements distincts. Dans main(), tigre et perroquet sont tous les deux déclarés de type Animal, mais tigre.faireDuBruit() affiche "Tigre grogne." et perroquet.faireDuBruit() affiche "Perroquet chante.". C'est le polymorphisme d'exécution (late binding).

2.2 Partie 2 — Code complété

Question 5 : Méthode voler() dans la classe Oiseau

```java
public void voler() {
    System.out.println("L'oiseau vole.");
}
```

Question 6 : Utilisation de voler() dans main()

```java
// Cast en Oiseau pour accéder à la méthode voler()
Oiseau perroquetOiseau = new Oiseau("Perroquet");
perroquetOiseau.faireDuBruit(); // Sortie : Perroquet chante.
perroquetOiseau.voler();        // Sortie : L'oiseau vole.
```

2.3 Partie 3 — Identification des mots-clés

Concept	Mot-clé Java	Exemple dans le code
Héritage	`extends`	`class Mammifere extends Animal`
Redéfinition de méthode	`@Override`	`@Override public void faireDuBruit()`
Abstraction	`abstract`	`abstract class Animal / abstract void faireDuBruit()`

3. Exercice 1.2 — Gestion des véhicules

Ce deuxième exercice modélise un système de gestion de véhicules avec une classe abstraite Vehicule et les sous-classes Voiture, Camion et Moto (ajoutée en Q6).

3.1 Partie 1 — Réponses théoriques

Q1. Encapsulation :

> Les attributs marque, modele et annee sont déclarés private dans la classe Vehicule. Ils ne sont accessibles depuis l'extérieur que via les getters getMarque(), getModele() et getAnnee(). Cela protège les données et permet de contrôler leur accès.

Q2. Héritage :

> Voiture extends Vehicule et Camion extends Vehicule illustrent l'héritage. Les deux classes héritent des attributs marque, modele, annee et des getters associés. Elles appellent super(marque, modele, annee) dans leur constructeur pour initialiser les attributs de la classe parente.

Q3. Polymorphisme :

> Dans main(), maVoiture et monCamion sont déclarés de type Vehicule. L'appel maVoiture.afficherDetails() exécute la version de Voiture, et monCamion.afficherDetails() exécute la version de Camion. La méthode exécutée est déterminée à l'exécution selon le type réel de l'objet.

Q4. Abstraction :

> Vehicule est abstraite car elle définit un contrat (méthode abstraite afficherDetails()) sans l'implémenter. On ne peut pas créer un objet Vehicule directement car un véhicule générique n'a pas de sens — seules les sous-classes concrètes (Voiture, Camion, Moto) peuvent être instanciées.

3.2 Partie 2 — Code complété

Question 5 : Méthode demarrer() et modification de afficherDetails()

```java
// Dans la classe Vehicule :
public void demarrer() {
    System.out.println("Le véhicule démarre.");
}

// Dans Voiture.afficherDetails() :
@Override
public void afficherDetails() {
    demarrer(); // appel à la méthode parente
    System.out.println("Voiture: " + getMarque() + " " + getModele()
        + " (" + getAnnee() + "), Portes: " + nombreDePortes);
}
```

Question 6 : Nouvelle classe Moto

```java
class Moto extends Vehicule {
    private String typeDeGuidon;

    public Moto(String marque, String modele, int annee, String typeDeGuidon) {
        super(marque, modele, annee);
        this.typeDeGuidon = typeDeGuidon;
    }

    @Override
    public void afficherDetails() {
        demarrer();
        System.out.println("Moto: " + getMarque() + " " + getModele()
            + " (" + getAnnee() + "), Guidon: " + typeDeGuidon);
    }
}
```

3.3 Partie 3 — Identification des mots-clés

Concept	Mot-clé Java	Exemple dans le code

Redéfinition de méthode	`@Override`	`@Override public void afficherDetails()`
Héritage	`extends`	`class Voiture extends Vehicule`
Appel constructeur parent	`super(...)`	`super(marque, modele, annee)`

4. Exercice 1.3 — Gestion des comptes bancaires

Ce troisième exercice implémente un système de gestion bancaire avec une classe de base CompteBancaire et deux classes dérivées : CompteCourant et CompteEpargne.

4.1 Implémentation de CompteBancaire

Java
```java
class CompteBancaire {
    private String numeroCompte;
    private double solde;

    public CompteBancaire(String numeroCompte, double soldeInitial) {
        this.numeroCompte = numeroCompte;
        this.solde        = soldeInitial;
    }

    public void deposer(double montant) {
        if (montant > 0) { solde += montant; }
    }

    public void retirer(double montant) {
        if (montant > solde) {
            System.out.println("Solde insuffisant.");
        } else {
            solde -= montant;
        }
    }
}
```

4.2 Implémentation de CompteEpargne

Java
```java
class CompteEpargne extends CompteBancaire {
    public CompteEpargne(String numeroCompte, double soldeInitial) {
        super(numeroCompte, soldeInitial);
    }

    public void calculerInterets(double taux) {
        double interets = getSolde() * taux / 100;
        deposer(interets);
        System.out.println("Intérêts (" + taux + "%) : +" + interets + " €");
    }
}
```

4.3 Implémentation de CompteCourant

Java

```java
class CompteCourant extends CompteBancaire {
    private double decouvertAutorise;

    public CompteCourant(String num, double solde, double decouvert) {
        super(num, solde);
        this.decouvertAutorise = decouvert;
    }

    @Override
    public void retirer(double montant) {
        if (montant > getSolde() + decouvertAutorise) {
            System.out.println("Dépassement du découvert autorisé.");
        } else {
            super.retirer(montant);
        }
    }
}
```

4.4 Classe principale Banque — Résultats d'exécution

```java
====== COMPTE COURANT ======
Compte [CC-001] — Solde : 1000.0 €
Dépôt de 200.0 € effectué. Solde : 1200.0 €
Retrait de 300.0 € effectué. Solde : 900.0 €
Dépassement du découvert autorisé (500.0 €).
Compte [CC-001] — Solde : 900.0 €

====== COMPTE ÉPARGNE ======
Compte [CE-001] — Solde : 5000.0 €
Dépôt de 1000.0 € effectué. Solde : 6000.0 €
Retrait de 200.0 € effectué. Solde : 5800.0 €
Intérêts calculés (3.5%) : +203.0 €. Nouveau solde : 6003.0 €
Compte [CE-001] — Solde : 6003.0 €
```

5. Synthèse des concepts POO

Concept	Définition	Exemple dans le TP
Encapsulation	Cacher les attributs (private) et les exposer via des getters/setters	Attribut nom dans Animal, attributs marque/modele dans Vehicule
Héritage	Une classe fille réutilise les attributs et méthodes d'une classe parente (extends)	Mammifere extends Animal, Voiture extends Vehicule, CompteEpargne extends CompteBancaire
Abstraction	Définir une structure générale (abstract) sans implémenter tous les détails	Classes Animal et Vehicule abstraites, méthodes abstraites faireDuBruit() et afficherDetails()
Polymorphisme	Même méthode, comportements différents selon le type réel de l'objet (@Override)	tigre.faireDuBruit() vs perroquet.faireDuBruit(), maVoiture.afficherDetails() vs monCamion.afficherDetails()

6. Conclusion

Ce TP nous a permis de maîtriser les quatre piliers fondamentaux de la Programmation Orientée Objet en Java. À travers les trois exercices, nous avons appliqué l'encapsulation pour protéger les données, l'héritage pour réutiliser le code, l'abstraction pour définir des contrats entre classes, et le polymorphisme pour écrire du code flexible et extensible. Ces concepts sont essentiels pour concevoir des applications Java robustes et maintenables.